



BitTTS: Highly Compact Text-to-Speech Using 1.58-bit Quantization and Weight Indexing

Masaya Kawamura¹, Takuya Hasumi¹, Yuma Shirahata¹, Ryuichi Yamamoto¹

¹LY Corporation, Japan

kawamura.masaya@lycorp.co.jp, takuya.hasumi@lycorp.co.jp, yuma.shirahata@lycorp.co.jp,
ryuichi.yamamoto@lycorp.co.jp

Abstract

This paper proposes a highly compact, lightweight text-to-speech (TTS) model for on-device applications. To reduce the model size, the proposed model introduces two techniques. First, we introduce quantization-aware training (QAT), which quantizes model parameters during training to as low as 1.58-bit. In this case, most of 32-bit model parameters are quantized to ternary values $\{-1, 0, 1\}$. Second, we propose a method named weight indexing. In this method, we save a group of 1.58-bit weights as a single int8 index. This allows for efficient storage of model parameters, even on hardware that treats values in units of 8-bit. Experimental results demonstrate that the proposed method achieved 83 % reduction in model size, while outperforming the baseline of similar model size without quantization in synthesis quality.

Index Terms: Text-to-speech, quantization-aware training, lightweight

1. Introduction

Many high-quality neural text-to-speech (TTS) models have been extensively researched [1–5]. With the advancement of these TTS models, they are increasingly being integrated into mobile applications, such as car navigation systems and conversational bots, among others. In these applications, TTS models are typically required to generate speech quickly on low-end devices with limited memory and storage. Therefore, research on 1) fast inference with limited computational resources and 2) model size compression¹ is crucial for ensuring that TTS technology can be effectively used in many real-world applications.

To address the former, various lightweight TTS models have been proposed and demonstrated promising results [6–11]. The methods include multi-band generation [9] and convolution-based lightweight blocks [8], as well as other approaches. On the other hand, for the model size compression of lightweight TTS models, research in this area is still in its early stages. Although some studies have tackled this through knowledge distillation [7, 10, 11] or neural architecture search [6, 12, 13], there remains room for further investigation.

One approach to reduce the model size of neural network models is quantization [14, 15]. Quantization reduces the model size by representing weights with low precision. According to [14, 15], this can be roughly categorized into post-training quantization (PTQ) and quantization-aware training (QAT). PTQ involves quantizing a pre-trained model without further training, which can lead to significant degradation in model performance due to the abrupt reduction in numerical precision. In contrast, QAT incorporates quantization into the training process, allowing the model to adapt to the lower precision weights. This

enables QAT to mitigate the accuracy loss typically associated with quantization, as confirmed by previous studies in fields such as computer vision [14, 16].

In addition, while many studies have investigated 8-bit or 4-bit quantization of neural network models [17, 18], some recent studies report that extremely low precision such as 1 or 1.58-bit still works well in tasks such as language modeling [19, 20] and computer vision [21–25]. Despite the success of QAT in many fields, it has not been extensively studied in the context of lightweight TTS.

In this paper, we propose a lightweight TTS model with 1.58-bit quantization to reduce the size of TTS models for on-device applications. Since QAT has not been investigated in the TTS field so far, we also evaluate 4-bit variants to explore the trade-offs between model size and synthesized speech quality. In the 1.58-bit quantization setting, the model parameters are quantized to ternary values $\{-1, 0, 1\}$. This setting is expected to significantly reduce the size of each model weight, which is otherwise represented by 32-bit (float32). However, it is typically difficult to save the quantized weights with their original amount of information (i.e., save each weight as 1.58-bit) due to hardware limitations, which handle all weights as 8-bit or 16-bit units. To overcome this limitation, we propose a method named *weight indexing*, which saves the model weights as indices of a predefined set of *weight patterns*. Specifically, since the total number of 5 weight combinations is at most $3^5 = 243$, every 5 weights are stored as a single 8-bit integer, which can represent $2^8 = 256$ patterns². This enables saving the weights at a size close to the original amount of information. We call the TTS model with quantization and weight indexing as *BitTTS*.

Experimental results on lightweight TTS models showed that the proposed method with weight indexing can reduce the model size up to 83 % when both the acoustic model and the vocoder are quantized. In terms of the synthesis quality, the model outperformed a baseline model of similar model size without quantization. We also found that avoiding the quantization of the vocoder significantly improves the synthesized speech quality. Even in this case, the model size was still reduced by about 70 %. Audio samples are available on our demo page³.

2. Method

2.1. Quantization-aware training

QAT [16] is a technique to quantize models dynamically during training. This quantization approach is widely used in not only

¹In addition to the storage usage, model size compression is also beneficial for TTS application users to save mobile data usage.

²We use 5 here because 243 patterns can be efficiently represented by the 256 patterns supported by an 8-bit integer. In other words, we store $1.58 \times 5 = 7.9$ bit into an 8-bit integer.

³<https://masayakawamura.github.io/bittts-demo/>

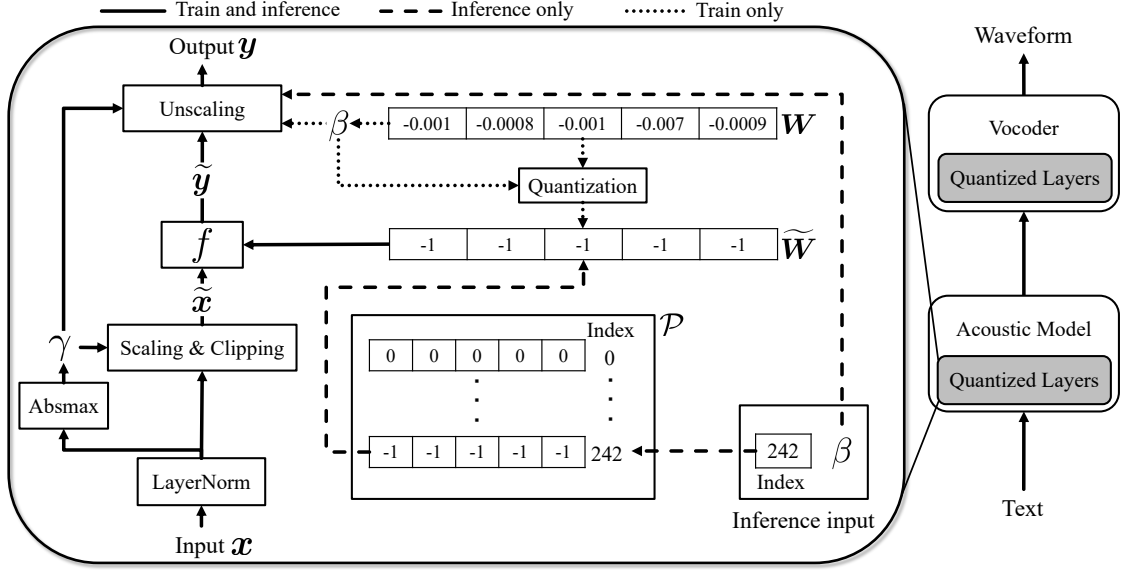


Figure 1: *BitTTS* quantizes the weight of TTS. The proposed method reduces the model size by storing indices instead of weights.

natural language processing [19, 20] but also speech recognition [17, 26] due to its higher accuracy [14, 16] than the static quantization such as PTQ. In this paper, we applied a popular QAT method called fake QAT [16].

2.2. Quantization formulation

The weights of a neural network layer are denoted as $\mathbf{W} = \{w_1, w_2, \dots, w_N\}$, where w_i denotes i -th scalar weight, and N represents the total number of weights. Inspired by the BitNet b1.58 method [20], we scale the weights $\mathbf{W}$ by their average absolute values and then round each value to the nearest integer. When quantizing $\mathbf{W}$ to b -bit, the following equations are used:

$$\tilde{\mathbf{W}} = \text{Round} \left(\text{Clip} \left(\frac{\mathbf{W}}{\beta + \epsilon}, -q, q - 1 \right) \right), \quad (1)$$

$$q = 2^{(b-1)}, \quad (2)$$

$$\text{Clip}(u, l, h) = \max(l, \min(h, u)). \quad (3)$$

Specifically, when quantizing $\mathbf{W}$ to 1.58-bit, it is quantized to ternary values $\{-1, 0, 1\}$:

$$\tilde{\mathbf{W}} = \text{Round} \left(\text{Clip} \left(\frac{\mathbf{W}}{\beta + \epsilon}, -1, 1 \right) \right), \quad (4)$$

where ϵ is a small floating-point number that prevents division by zero. β is calculated as follows:

$$\beta = \frac{1}{N} \sum_{w \in \mathbf{W}} |w|. \quad (5)$$

In the case of convolutional layers, the weights $\mathbf{W}$ depend on the number of output channels c_{out} , the number of input channels c_{in} , and the kernel size K . The weights are represented as $\mathbf{W} \in \mathbb{R}^{c_{\text{out}} \times c_{\text{in}} \times K}$, and β is calculated as follows:

$$\beta = \frac{1}{c_{\text{out}} c_{\text{in}} K} \sum_{i=1}^{c_{\text{out}}} \sum_{j=1}^{c_{\text{in}}} \sum_{k=1}^K |\mathbf{W}_{i,j,k}|, \quad (6)$$

where $\mathbf{W}_{i,j,k}$ represents the weight corresponding to the i -th output channel, the j -th input channel and the k -th kernel index, respectively.

Layer normalization [27] is applied to the input tensor $\mathbf{x}$. After the layer normalization, the input $\mathbf{x}$ is scaled to the range $[-Q_p, Q_p]$ where $Q_p = 2^{p-1}$, with p representing p -bit precision. The scaling and clipping are performed as follows:

$$\tilde{\mathbf{x}} = \text{Clip} \left(\frac{\mathbf{x} Q_p}{\gamma}, -Q_p + \epsilon, Q_p - \epsilon \right), \quad (7)$$

$$\gamma = \|\mathbf{x}\|_{\infty}. \quad (8)$$

Here, γ is the infinity norm of the input tensor $\mathbf{x}$, representing the maximum absolute value within $\mathbf{x}$.

The output can be obtained from the quantized weights $\tilde{\mathbf{W}}$ and input $\tilde{\mathbf{x}}$. Let f represent the neural network operation, then it can be written as follows:

$$\tilde{\mathbf{y}} = f(\tilde{\mathbf{W}}, \tilde{\mathbf{x}}). \quad (9)$$

For example, in the case of a convolutional layer, f represents the convolution operation. To restore the original scale, the output $\tilde{\mathbf{y}}$ is rescaled by multiplying it by the scale factors γ and β , and dividing by the quantization range Q_p . The rescaled output $\mathbf{y}$ can be expressed as:

$$\mathbf{y} = \frac{\tilde{\mathbf{y}} \gamma \beta}{Q_p}. \quad (10)$$

Following BitNet b1.58 [20], we do not use the bias. Due to the non-differentiable functions such as clipping and rounding, we apply the straight-through estimator [28] during backpropagation to approximate the gradient.

2.3. Weight indexing

If a model weight is quantized with ternary values $\{-1, 0, 1\}$, each value can be ideally represented by $\log_2 3 \approx 1.58$ -bit. However, since most deep learning frameworks or hardware typically handle values in units of 8-bit or 16-bit, quantization by ternary values does not effectively reduce the model size. This is because each 1.58-bit weight is saved as 8-bit or 16-bit value. To efficiently save weights, we store

Algorithm 1 Weight Indexing

```
1: Input:  $\tilde{\mathbf{v}} = (\tilde{v}_1, \dots, \tilde{v}_L)^\top \in \{-1, 0, 1\}^L$ , block size  $L^*$ 
2: Output: List of indices of weight pattern  $\mathcal{P}$ 
3: Initialize indices =  $\emptyset$ 
4: for each block in  $\{1, 2, \dots, \lceil L/L^* \rceil\}$  do
5:   Initialize  $n = 0$ 
6:   for each  $l$  in  $\{\min(\text{block} \times L^*, L), \dots, (\text{block} - 1) \times L^* + 1\}$  do
7:      $n \leftarrow n \times 3$ 
8:     if  $\tilde{v}_l = 1$  then
9:        $n \leftarrow n + 1$ 
10:    else if  $\tilde{v}_l = -1$  then
11:       $n \leftarrow n + 2$ 
12:    end if
13:  end for
14:  Append  $n$  to indices
15: end for
16: Return: List of indices
```

the integer indices of a predefined set of weight patterns $\mathcal{P} = \{(0, \dots, 0, 0)^\top, (0, \dots, 0, 1)^\top, \dots, (-1, \dots, -1, -1)^\top\}$ instead of quantized weights $\tilde{\mathbf{W}}$ itself.

Let us assume that the quantized weight $\tilde{\mathbf{W}}$ can be represented as a flattened vector $\tilde{\mathbf{v}} \in \mathbb{R}^L$ to generalize. Algorithm 1 outlines the steps to generate a list of weight patterns for a flattened vector $\tilde{\mathbf{v}}$. In this algorithm, we first define a block size L^* , by which the whole L weights are divided into segments of length L^* . Then, every L^* weight is mapped to a virtual lookup table representing weight patterns $\mathcal{P}$ with length 3^{L^*} . Finally, the list of weight patterns is saved.

In practice, to represent weight patterns as 8-bit integers, we choose $L^* = 5$. If $L^* = 5$, $3^{L^*} \approx 2^8$ holds, which leads to the effective use of the bit length in 8-bit. For example, in the case of 1D convolution with $c_{\text{in}} = c_{\text{out}} = 256$ and kernel size $K = 5$, the weight can be ideally saved as 63.4kB^4 when quantized to 1.58-bit. However, if the hardware only supports 8-bit as the minimum precision, the weight is saved as 320kB^5 . By using our algorithm, the quantized weight is saved as 64.0kB^6 , which shows the significant model size reduction.

Figure 1 shows the quantization of the TTS model using the proposed method. The stored indices are used to quickly reconstruct the quantized weights $\tilde{\mathbf{W}}$ from the predefined patterns $\mathcal{P}$. This involves retrieving the corresponding pattern for each index and concatenating the segments to form the complete weight tensor.

3. Experiments

3.1. Experimental conditions

We conducted experiments to evaluate the effectiveness of quantization in TTS and the proposed methods. We used LibriTTS-R [29], a high-quality multi-speaker corpus for experiments. This corpus comprises 585 hours of speech data at a 24 kHz sampling rate from 2,456 English speakers. We divided the corpus into a training set (475.61 hours), a validation set (0.17 hours), and a test set (51.92 hours). To train the duration model in TTS models, we extracted phone duration by Montreal forced aligner [30].

⁴ $(c_{\text{in}} \times c_{\text{out}} \times L^* \times \log_2 3) / (8 \times 2^{10}) = 63.4$

⁵ $(c_{\text{in}} \times c_{\text{out}} \times L^* \times 8) / (8 \times 2^{10}) = 320$

⁶ $(c_{\text{in}} \times c_{\text{out}} \times 8) / (8 \times 2^{10}) = 64.0$

To evaluate the effectiveness of the proposed method, we conducted experiments using a TTS model. As the base TTS architecture, we adopted JETS [31] for its simple architecture. However, since the transformer blocks [32] in the encoder and the decoder of JETS are computationally expensive, these blocks were substituted with 4 convolutional layers of kernel size 5. The number of channels was set to 256 for the encoder and 192 for the decoder. The vocoder used is HiFi-GAN [33]. The initial channel in HiFi-GAN was set to 128. Additionally, to confirm the effectiveness of the quantization for model size compression, we prepared another 32-bit baseline model. In this model, the model size was reduced using small channel sizes instead of quantization. For this model, the channel of the encoder and decoder in the acoustic model and the initial channel of HiFi-GAN were set to 32. We omit the energy predictor from these models for simplicity. Additionally, to improve naturalness, we added a mixture density network [34] layer to the duration predictor [35].

We used an NVIDIA A100 GPU to train all the models. We utilized AdamW optimizer [36] with $\beta_1 = 0.8$, $\beta_2 = 0.99$. The initial learning rate was 2×10^{-4} . The learning rate decay was scheduled using the exponential learning rate scheduler [37] with a decay factor of 0.9973. We used a dynamic batch size with an average of 15 to create a minibatch [38]. All models were trained for 2000 K steps.

3.2. Quantization details

For quantization, considering that more than 90 % of the model weights in the baseline TTS models belong to 1D convolutional layers, we applied quantization to only 1D convolutional layers in baseline TTS models for simplicity. Quantized weights in convolutional layers were stored as int8, while other weights were stored as float32. In the case of 4-bit quantization, the quantization method follows the approach in Section 2.2, with the clipping range in Equation 4 set to $-(2^{(4-1)}) = -8$ to $2^{(4-1)} - 1 = 7$. Additionally, p was set to 8. In the case of 1.58-bit, the quantization method follows the approach in Section 2.2, and the weight indexing from Section 2.3 was applied. Note that when quantizing the TTS vocoder to 4-bit or 1.58-bit, the convolutional layer closest to the waveform output was excluded from quantization. This is because preliminary experiments showed that quantizing this layer significantly degrades the quality of the synthesized speech. Additionally, to investigate the impact of quantizing different modules, we conducted experiments in three settings: quantization of 1) both the acoustic model and vocoder, 2) only the acoustic model, and 3) only the vocoder.

3.3. Evaluation

3.3.1. Naturalness of synthesized speech

To evaluate the quality of generated samples, we conducted a subjective listening test using five-point naturalness mean opinion score (MOS). We randomly selected 30 utterances from the test data and synthesized speech samples. Then, 15 raters listened to the samples for evaluation. The experimental results are shown in Table 1. The results indicate that the 32-bit small model, created by naively reducing the size of the 32-bit TTS model, significantly decreases the MOS compared with the base 32-bit TTS model. In contrast, the 4-bit and 1.58-bit TTS models achieve higher MOS compared to the smaller 32-bit model. This result indicates that quantization is effective for creating small TTS models without significantly degrading quality.

When comparing the 4-bit and 1.58-bit TTS models, the 1.58-bit models achieve comparable or even higher MOS de-

Table 1: Comparison of model size and naturalness MOS with 95 % confidence intervals and average RTF on an Apple M1 Pro.

Method	Quantization		Weight Indexing	Model Size [MB]	MOS	RTF
	Acoustic Model	Vocoder				
32-bit	-	-	-	25.66	3.75 ± 0.09	0.042
32-bit (small model)	-	-	-	12.78	1.20 ± 0.04	0.019
4-bit	✓	-	-	11.40	3.34 ± 0.10	0.047
4-bit	-	✓	-	23.12	2.57 ± 0.10	0.062
4-bit	✓	✓	-	8.87	2.96 ± 0.11	0.068
1.58-bit	✓	-	-	11.40	3.30 ± 0.10	0.040
	✓	-	✓	7.60		
1.58-bit	-	✓	-	23.12	3.18 ± 0.11	0.059
	-	✓	✓	22.45		
1.58-bit	✓	✓	-	8.87	3.09 ± 0.11	0.064
	✓	✓	✓	4.39		
Ground truth	-	-	-	-	4.24 ± 0.09	-

spite the reduced model size. Particularly, when the vocoder is quantized, there is a significant MOS gap. One possible reason could be the instability in vocoder training due to quantization, which might have resulted in the 4-bit vocoder not learning effectively in this experiment.

Comparing the modules to be quantized, the quantization of the vocoder results in lower MOS than that of the acoustic model. This could be because the vocoder, which generates the waveform, is more sensitive to quality degradation when quantized than the acoustic model. In addition, quantizing the acoustic model resulted in a greater reduction in the model size. This is because, in the TTS model used in this experiment, the majority of the model size is attributed to the acoustic model. Given these results, quantizing the acoustic model would be one of the best options in terms of the model size and the synthesis quality.

Furthermore, the 1.58-bit TTS models with weight indexing achieve a smaller model size than the 4-bit models and 1.58-bit TTS without weight indexing, reducing the model size by up to 83 %⁷ compared to the base 32-bit TTS model. This result demonstrates that weight indexing is highly effective in reducing model size. When the vocoder was not quantized for better synthesis quality, the model size was still reduced to 70 %.

3.3.2. Inference speed

We measured the average real-time factor (RTF) on an Apple M1 Pro CPU using 100 randomly sampled utterances from the test set. For the method using weight indexing, we first restored the weights from the index and then measured the RTF.

Regarding the RTF, the small 32-bit model marked the best score. This is because the complexity of the 32-bit small TTS model is the lowest by removing a significant portion of the parameters from the base 32-bit TTS model. Quantized models tend to be slightly slow due to additional layers and operations introduced by the quantization method described in Section 2.2. When the vocoder is quantized, this effect becomes more pronounced due to the vocoder’s high computational cost. In contrast, the processing of the acoustic model is more lightweight than the vocoder, resulting in minimal impact on RTF when quantized. Therefore, if inference speed is a priority, quantizing only the acoustic model would be the best choice.

⁷ $(25.66 - 4.39)/25.66 \times 100$

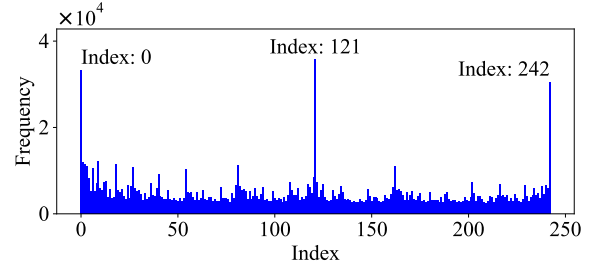


Figure 2: Frequency of indices when applying weight indexing to the 1.58-bit TTS model.

3.3.3. Analysis of weight patterns

Figure 2 shows the distribution of indices after applying weight indexing. The distribution is calculated from the 1.58-bit quantized TTS model with both the acoustic model and vocoder quantized. The result indicates that the indices are biased towards specific values and are not uniformly distributed. In particular, indices 0, 121, and 242 appear more frequently, corresponding to weight values of 0, 1, and -1 repeated L^* times, respectively. Future work will explore methods to further reduce the model size by leveraging this bias in the occurrence frequency of indices⁸.

4. Conclusions

In this paper, we proposed a highly compact TTS model with 1.58-bit quantization for on-device applications. Additionally, we introduced a method called weight indexing to further reduce model size. Since QAT has not been explored in the TTS field, we also evaluated the trade-off between model size and synthesized speech quality. Experimental results showed that our proposed method could reduce the model size by over 80 % while achieving a naturalness MOS comparable to or higher than that of the 4-bit TTS models. Furthermore, we found that quantization of the vocoder significantly affects the quality of synthesized speech in TTS models. Future work includes improving the quality of TTS using the proposed method and applying weight indexing to other models.

⁸After applying Huffman coding [39] to the model, which quantized both the acoustic model and vocoder to 1.58-bit and using weight indexing, the model size was reduced from 4.39 MB to 4.35 MB.

5. References

- [1] H. Zen, A. Senior, and M. Schuster, “Statistical parametric speech synthesis using deep neural networks,” in *Proc. ICASSP*, 2013, pp. 7962–7966.
- [2] Y. Ning, S. He, Z. Wu, C. Xing, and L.-J. Zhang, “A review of deep learning based speech synthesis,” *Appl. Sci.*, vol. 9, no. 19, 2019.
- [3] X. Tan, T. Qin, F. Soong, and T.-Y. Liu, “A survey on neural speech synthesis,” *arXiv preprint arXiv:2106.15561*, 2021.
- [4] A. Triantafyllopoulos, B. W. Schuller, G. İymen, M. Sezgin, X. He, Z. Yang, P. Tzirakis, S. Liu, S. Mertes, E. André, R. Fu, and J. Tao, “An overview of affective speech synthesis and conversion in the deep learning era,” *Proc. of the IEEE*, vol. 111, no. 10, pp. 1355–1381, 2023.
- [5] A. Mehrish, N. Majumder, R. Bharadwaj, R. Mihalcea, and S. Poria, “A review of deep learning techniques for speech processing,” *Information Fusion*, vol. 99, p. 101869, 2023.
- [6] R. Luo, X. Tan, R. Wang, T. Qin, J. Li, S. Zhao, E. Chen, and T.-Y. Liu, “Lightspeech: Lightweight and fast text to speech with neural architecture search,” in *Proc. ICASSP*, 2021, pp. 5699–5703.
- [7] R. Chevi, R. E. Prasoj, A. F. Aji, A. Tjandra, and S. Sakti, “NIX-TTS: Lightweight and end-to-end text-to-speech via module-wise distillation,” in *Proc. SLT*, 2023, pp. 970–976.
- [8] T. Okamoto, Y. Ohtani, T. Toda, and H. Kawai, “ConvNeXt-TTS and ConvNeXt-VC: ConvNeXt-based fast end-to-end sequence-to-sequence text-to-speech and voice conversion,” in *Proc. ICASSP*, 2024, pp. 12 456–12 460.
- [9] M. Kawamura, Y. Shirahata, R. Yamamoto, and K. Tachibana, “Lightweight and high-fidelity end-to-end text-to-speech with multi-band generation and inverse short-time fourier transform,” in *Proc. ICASSP*, 2023, pp. 1–5.
- [10] J. Vainer and O. Dušek, “SpeedySpeech: Efficient neural speech synthesis,” in *Proc. Interspeech*, 2020, pp. 3575–3579.
- [11] W. Ping, K. Peng, and J. Chen, “ClariNet: Parallel wave generation in end-to-end text-to-speech,” in *Proc. ICLR*, 2019.
- [12] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, “Learning transferable architectures for scalable image recognition,” in *Proc. CVPR*, 2018.
- [13] R. Luo, F. Tian, T. Qin, E. Chen, and T.-Y. Liu, “Neural architecture optimization,” in *Proc. NeurIPS*, vol. 31, 2018.
- [14] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A survey of quantization methods for efficient neural network inference,” *arXiv preprint arXiv:2103.13630*, 2021.
- [15] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, “A white paper on neural network quantization,” *arXiv preprint arXiv:2106.08295*, 2021.
- [16] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. CVPR*, 2018, pp. 2704–2713.
- [17] S. Ding, P. Meadowlark, Y. He, L. Lew, S. Agrawal, and O. Rybakov, “4-bit conformer with native quantization aware training for speech recognition,” in *Proc. Interspeech*, 2022, pp. 1711–1715.
- [18] K. Zhen, H. D. Nguyen, R. Chinta, N. Susanj, A. Mouchtaris, T. Afzal, and A. Rastrow, “Sub-8-bit quantization aware training for 8-bit neural network accelerator with on-device speech recognition,” in *Proc. Interspeech*, 2022, pp. 3033–3037.
- [19] H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei, “Bitnet: Scaling 1-bit transformers for large language models,” *arXiv preprint arXiv:2310.11453*, 2023.
- [20] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, “The era of 1-bit llms: All large language models are in 1.58 bits,” *arXiv preprint arXiv:2402.17764*, 2024.
- [21] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, “Xnor-net: Imagenet classification using binary convolutional neural networks,” in *Proc. ECCV*, 2016, pp. 525–542.
- [22] A. Bulat and G. Tzimiropoulos, “Xnor-net++: Improved binary neural networks,” in *Proc. BMVC*, 2019.
- [23] H. Qin, R. Gong, X. Liu, X. Bai, J. Song, and N. Sebe, “Binary neural networks: A survey,” *Pattern Recognition*, vol. 105, p. 107281, 2020.
- [24] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, “Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1,” *arXiv preprint arXiv:1602.02830*, 2016.
- [25] B. Xia, Y. Zhang, Y. Wang, Y. Tian, W. Yang, R. Timofte, and L. Van Gool, “Basic binary convolution unit for binarized image restoration network,” in *Proc. ICLR*, 2023.
- [26] O. Rybakov, P. Meadowlark, S. Ding, D. Qiu, J. Li, D. Rim, and Y. He, “2-bit conformer quantization for automatic speech recognition,” in *Proc. Interspeech*, 2023, pp. 4908–4912.
- [27] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv:1607.06450*, 2016.
- [28] Y. Bengio, N. Léonard, and A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” *arXiv preprint arXiv:1308.3432*, 2013.
- [29] Y. Koizumi, H. Zen, S. Karita, Y. Ding, K. Yatabe, N. Morioka, M. Bacchiani, Y. Zhang, W. Han, and A. Bapna, “LibriTTS-R: A restored multi-speaker text-to-speech corpus,” in *Proc. Interspeech*, 2023, pp. 5496–5500.
- [30] M. McAuliffe, M. Socolof, S. Mihuc, M. Wagner, and M. Sonderegger, “Montreal forced aligner: Trainable text-speech alignment using kald,” in *Proc. Interspeech*, vol. 2017, 2017, pp. 498–502.
- [31] D. Lim, S. Jung, and E. Kim, “JETS: Jointly training FastSpeech2 and HiFi-GAN for end to end text to speech,” in *Proc. Interspeech*, 2022, pp. 21–25.
- [32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proc. NeurIPS*, 2017, pp. 5998–6008.
- [33] J. Kong, J. Kim, and J. Bae, “HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis,” in *Proc. NeurIPS*, vol. 33, 2020, pp. 17 022–17 033.
- [34] C. M. Bishop, “Mixture density networks,” *Aston University, Birmingham UK*, 1994.
- [35] C. Du and K. Yu, “Phone-level prosody modelling with GMM-based MDN for diverse and controllable speech synthesis,” *IEEE/ACM Trans. on Audio, Speech, and Lang. Process.*, vol. 30, pp. 190–201, 2022.
- [36] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.
- [37] Z. Li and S. Arora, “An exponential learning rate schedule for deep learning,” *arXiv preprint arXiv:1910.07454*, 2019.
- [38] T. Hayashi, R. Yamamoto, K. Inoue, T. Yoshimura, S. Watanabe, T. Toda, K. Takeda, Y. Zhang, and X. Tan, “ESPnet-TTS: Unified, reproducible, and integratable open source end-to-end text-to-speech toolkit,” in *Proc. ICASSP*, 2020, pp. 7654–7658.
- [39] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.